

Target SQL Analysis – Project Submission

S Sathish Kumar

Sathishkumar19@gmail.com

Table of Contents

TARGET SQL ANALYSIS – PROJECT SUBMISSION.....	1
A. DATASET	2
1. Dataset CSV Files	2
2. Business Case	2
B. DATASET UPLOADING TO BIGQUERY	2
3. Importing the Data to Big Query	2
4. DataSet Upload to Big_Query – Output	4
5. Validation of SQL Query.....	4
C. EXPLORATORY DATA ANALYSIS - EDA.....	4
6. List of Tables.....	5
7. Checking the size of dataset/tables	5
8. Volume of Data in Each Table	6
D. BUSINESS CASE STUDY TASKS	8
1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset: 8	
1.1 Data type of all columns in the “customers” table.....	8
1.2 Get the time range between which the orders were placed.....	9
1.3 Count the Cities & States of customers who ordered during the given period.....	9
2. In-depth Exploration:.....	10
2.1 Is there a growing trend in the no. of orders placed over the past years?.....	10
2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?.....	11
2.3 During what time of the day, do the Brazilian customers mostly place their orders?	13
3. Evolution of E-commerce orders in the Brazil region:.....	14
3.1 Get the month on month no. of orders placed in each state.....	14
3.2 How are the customers distributed across all the states?	16
4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.....	17
4.1 Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).	17
4.2 Calculate the Total & Average value of order price for each state.....	19
4.3 Calculate the Total & Average value of order freight for each state.	21
5. Analysis based on sales, freight and delivery time.	22
5.1 Find the no. of days taken to deliver each order from the order’s purchase date as delivery time.	22
5.2 Find out the top 5 states with the highest & lowest average freight value.....	24
5.3 Find out the top 5 states with the highest & lowest average delivery time.	26
5.4 Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery....	28
6. Analysis based on the payments:	29
6.1 Find the month on month no. of orders placed using different payment types.....	29
6.2 Find the no. of orders placed on the basis of the payment instalments that have been paid.....	30
E. OVERALL INSIGHTS – KEY TAKEAWAYS	31

A. Dataset

1. Dataset CSV Files

Dataset is provided and is downloaded from the following link

<https://drive.google.com/drive/folders/1TGEc66YKbD443nslRi1bWgVd238gJCnb>

2. Business Case

Business case is described in the following document .



B. Dataset uploading to BigQuery

3. Importing the Data to Big Query

Since there are multiple CSV Files, I have used python Script to upload multiple CSV at ONCE. The source of all code is located in the following google colab Notebook

[Click Here : Google Colab Notebook – Target – Busines Case -SQL](#)

or

https://colab.research.google.com/drive/1SfuO3JIDonvXPV0IUo_U3wNWDYGXmsOy?usp=sharing

```
!pip install google-cloud-bigquery

# ✅ Create datasets and load CSVs
import os
import zipfile # Import zipfile
from google.cloud import bigquery
client = bigquery.Client(project=project_id)
# ✅ Ensure the base directory exists
os.makedirs(base_extract_path, exist_ok=True) # Create the directory if it doesn't exist
# ✅ Extract the uploaded ZIP file
for filename, content in uploaded.items():
    with open(os.path.join(base_extract_path, filename), 'wb') as f:
        f.write(content)
    with zipfile.ZipFile(os.path.join(base_extract_path, filename), 'r') as zip_ref:
        zip_ref.extractall(base_extract_path) # Extract to the base path
for dataset_id, subfolder in dataset_mapping.items():
    dataset_ref = bigquery.Dataset(f"{project_id}.{dataset_id}")
    try:
        client.get_dataset(dataset_ref)
        print(f"✅ Dataset already exists: {dataset_id}")
    except Exception:
        client.create_dataset(dataset_ref)
        print(f"🆕 Created dataset: {dataset_id}")
```

```

# ✅ Find the actual directory where CSV files are extracted
folder_path = base_extract_path # Start with the base path
for root, dirs, files in os.walk(base_extract_path):
    for file in files:
        if file.endswith(".csv"):
            folder_path = root # Update folder_path if a CSV is found
            break
    else:
        continue
    break # Break out of the outer loop as soon as a CSV is found
for file in os.listdir(folder_path):
    # Check if the file ends with .csv
    if file.endswith(".csv"):
        # Extract table_id from the filename by removing .csv and prefix
        table_id = file.replace(".csv", "").split("_", 1)[-1]
        file_path = os.path.join(folder_path, file)
        table_ref = f"{project_id}.{dataset_id}.{table_id}"
        job_config = bigquery.LoadJobConfig(
            source_format=bigquery.SourceFormat.CSV,
            skip_leading_rows=1,
            autodetect=True
        )
        with open(file_path, "rb") as source_file:
            job = client.load_table_from_file(source_file, table_ref, job_config=job_config)
        job.result() # Wait for the job to complete
        print(f"📁 Uploaded: {file} → {table_ref}")
print("✅ All CSV files uploaded to BigQuery.")

```

4. DataSet Upload to Big_Query – Output

```
Requirement already satisfied: google-cloud-bigquery in /usr/local/lib/python3.11/dist-packages (3.31.0)
Requirement already satisfied: google-api-core<3.0.0,>=2.11.1 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: google-auth<3.0.0,>=2.14.1 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: google-cloud-core<3.0.0,>=2.4.1 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: google-resumable-media<3.0.0,>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: packaging>=24.2.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: python-dateutil<3.0.0,>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: requests<3.0.0,>=2.21.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.56.2 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: protobuf!=3.20.0,!<3.20.1,!>4.21.0,!>4.21.1,!>4.21.2,!>4.21.3,!>4.21.4,!>4.21.5,<7.0.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: grpcio<2.0dev,>=1.33.2 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: grpcio-status<2.0dev,>=1.33.2 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: cachetools<6.0,>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: google-crc32c<2.0dev,>=1.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigquery)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil<3.0.0,>=2.8.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3.0.0,>=2.21.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3.0.0,>=2.21.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3.0.0,>=2.21.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3.0.0,>=2.21.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.11/dist-packages (from pyasn1-modules>=0.2.1)
Dataset already exists: Target_DB
Uploaded: order_reviews.csv → sathish-scaler-projects.Target_DB.order_reviews
Uploaded: sellers.csv → sathish-scaler-projects.Target_DB.sellers
Uploaded: products.csv → sathish-scaler-projects.Target_DB.products
Uploaded: geolocation.csv → sathish-scaler-projects.Target_DB.geolocation
Uploaded: order_items.csv → sathish-scaler-projects.Target_DB.order_items
Uploaded: orders.csv → sathish-scaler-projects.Target_DB.orders
Uploaded: customers.csv → sathish-scaler-projects.Target_DB.customers
Uploaded: payments.csv → sathish-scaler-projects.Target_DB.payments
All CSV files uploaded to BigQuery.
```

5. Validation of SQL Query

```
[ ] query = """
SELECT *
FROM `sathish-scaler-projects.Target_DB.customers`
LIMIT 20
"""
query_job = client.query(query)
results = query_job.result()
df = results.to_dataframe()
print(tabulate(df, headers='keys', tablefmt='psql'))
```

	customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
0	2201362e68992f654942dc0067c1b716	f7d7fc0a59ef4363fdce6e3aa069d498	69900	rio branco	AC
1	31dbcb13addc753e210692eacaea065e4	5dbba6c01268a8ad43f79157bf4454a0	69900	rio branco	AC
2	dad907e170748a35ef4e92238b7308f3	36b1c0516f123351ffa87430416dcae5	69900	rio branco	AC
3	888d2ebe1af2a8c93c75dae5dfc23719	721d1092e1a6460c67e6a0e691d899a3	69900	rio branco	AC
4	8a0108267d9258a0ec9f74381bc9b0de	7a2dc4682890550ebe3b8befcea3d55c	69900	rio branco	AC
5	5880e46677c68394bda62479fd673340	5dbba6c01268a8ad43f79157bf4454a0	69900	rio branco	AC
6	53996870173a3a001a1fb56ef00b9150	2624230437101e0bdcea1e48310d68a3	69900	rio branco	AC
7	cd281c1a7d26cd29a3ed4b029fce7270	086d6b5b5ba195a91aa0a6ec8e75d1a4	69900	rio branco	AC
8	717a3a9459f9006f23e24a684be8408b	5203034a29c3d294713bf8ccbd5d05a6	69900	rio branco	AC
9	2d618e470c95c9b425cbb0cbc42683dd	f7d7fc0a59ef4363fdce6e3aa069d498	69900	rio branco	AC
10	f34e32b8604090d1c2a7f3b04c834807	785a8a410d6fc9661d66f7218a87229d	69900	rio branco	AC
11	10ad09201fcc1c82d181ff7234bcd3b3	94742cd1fbac9146be7e2a139b63e13c	69900	rio branco	AC
12	929cb7b1618f0d99c7814d79b0c5b7cc	3cd07f2cd0e3bd52f3896c0048fb37c7	69900	rio branco	AC
13	0f32385df13e46d88d997460208bc866	4f67110f6d6d124111167b141bfa780	69900	rio branco	AC
14	6b4254cccae427caccdd90c738b60f31	2dfb04c19b2c957a47ff3ca8c1205349	69901	rio branco	AC
15	2d02f670df520fb5c5a818a9486c47	193c557222b982733aa8b9fa483a6c5e	69901	rio branco	AC
16	ded450170efe5531454485ca13120aca	c990f557eb1c943d3298d296015b9df4	69901	rio branco	AC
17	d7b7917cfa60997e23f7bb56c0055729	a73a341aa868b255594a9857c8e59772	69901	rio branco	AC
18	30e7b476534296021a9f7e0c289c6a86	12e92c0f870fc6941c2bfafe6357c4b	69901	rio branco	AC
19	7556392e5dea50250680dc5e7674cc7d	36db357344c8891e84f88e8d3ae4c538	69901	rio branco	AC

C. Exploratory Data Analysis - EDA

Getting used of datasets, now onwards, the outputs will be switched to Big query console for better readability

6. List of Tables

SQL Query

```
SELECT
    table_schema,
    table_name,
    column_name,
    data_type
FROM `Target_DB.INFORMATION_SCHEMA.COLUMNS`
ORDER BY
    table_schema, table_name, ordinal_position;
```

Output

Query results				
Job information Results Chart JSON Execution details Execution graph				
Row	table_schema	table_name	column_name	data_type
1	Target_DB	customers	customer_id	STRING
2	Target_DB	customers	customer_unique_id	STRING
3	Target_DB	customers	customer_zip_code_prefix	INT64
4	Target_DB	customers	customer_city	STRING
5	Target_DB	customers	customer_state	STRING
6	Target_DB	geolocation	geolocation_zip_code_prefix	INT64
7	Target_DB	geolocation	geolocation_lat	FLOAT64
8	Target_DB	geolocation	geolocation_lng	FLOAT64
9	Target_DB	geolocation	geolocation_city	STRING
10	Target_DB	geolocation	geolocation_state	STRING
11	Target_DB	items	order_id	STRING
12	Target_DB	items	order_item_id	INT64
13	Target_DB	items	product_id	STRING
14	Target_DB	items	seller_id	STRING
15	Target_DB	items	shipping_limit_date	TIMESTAMP
16	Target_DB	items	price	FLOAT64
17	Target_DB	items	freight_value	FLOAT64
18	Target_DB	orders	order_id	STRING
19	Target_DB	orders	customer_id	STRING
20	Target_DB	orders	order_status	STRING
21	Target_DB	orders	order_purchase_timestamp	TIMESTAMP
22	Target_DB	orders	order_approved_at	TIMESTAMP
23	Target_DB	orders	order_delivered_carrier_date	TIMESTAMP
24	Target_DB	orders	order_delivered_customer_date	TIMESTAMP
25	Target_DB	orders	order_estimated_delivery_date	TIMESTAMP
26	Target_DB	payments	order_id	STRING
27	Target_DB	payments	payment_sequential	INT64

Insights

- Since there is a lack of ER diagram, I will use this output for quick mapping of columns when writing subsequent queries.
- `INFORMATION_SCHEMA.COLUMNS` provides the information about all tables and their corresponding column names

7. Checking the size of dataset/tables

SQL Query

```

SELECT
    table_name,
    COUNT(*) as column_length
FROM
    `sathish-scaler-projects`.`Target_DB`.`INFORMATION_SCHEMA.COLUMNS`
GROUP BY table_name;

```

Output

Query results					
Job information		Results	Chart	JSON	Execution details
Execution graph					
Row	table_name	column_length			
1	sellers	4			
2	geolocation	5			
3	reviews	6			
4	products	9			
5	orders	8			
6	payments	5			
7	items	7			
8	customers	5			

Insights

- The output matches the list of CSV provided in the spreadsheet as per the business case,
- It also provides the no. of columns that we are dealing with at a quick glance

8. Volume of Data in Each Table

SQL Query

```

WITH
    cu_count AS (SELECT COUNT(*) AS customer_count FROM `sathish-scaler-projects.Target_DB.customers`),
    prod_count AS (SELECT COUNT(*) AS products_count FROM `sathish-scaler-projects.Target_DB.products`),
    se_count AS (SELECT COUNT(*) AS sellers_count FROM `sathish-scaler-projects.Target_DB.sellers`),
    ord_count AS (SELECT COUNT(*) AS orders_count FROM `sathish-scaler-projects.Target_DB.orders`),
    itm_count AS (SELECT COUNT(*) AS items_count FROM `sathish-scaler-projects.Target_DB.items`),
    pay_count AS (SELECT COUNT(*) AS payments_count FROM `sathish-scaler-projects.Target_DB.payments`),
    geo_count AS (SELECT COUNT(*) AS geolocation_count FROM `sathish-scaler-projects.Target_DB.geolocation`),
    rev_count AS (SELECT COUNT(*) AS reviews_count FROM `sathish-scaler-projects.Target_DB.reviews`)
SELECT
    customer_count,
    products_count,
    sellers_count,
    orders_count,
    items_count,
    payments_count,
    geolocation_count,
    reviews_count
FROM

```

```
cu_count, prod_count, se_count, ord_count, itm_count, pay_count, geo_count, rev_count;
```

Output

Query results

Job information		Results	Chart	JSON	Execution details	Execution graph			
Row		customer_count	products_count	sellers_count	orders_count	items_count	payments_count	geolocation_count	reviews_count
1		99441	32951	3095	99441	112650	103886	1000163	99224

Insights

- The output provides the volume of data that we are dealing with
- It also provides the view of no of customers, orders , reviews, products etc for the data set in a given period of time
- It also helps to provide a view point of which table is heavy in-terms of data count

D. Business Case Study tasks

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1.1 Data type of all columns in the “customers” table.

SQL Query

```
SELECT
  column_name,
  data_type
FROM
  `sathish-scaler-projects.Target_DB.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'customers';
```

Output

Query results

Job information	Results	Chart	JSON	Execution details
Row	column_name	data_type		
1	customer_id	STRING		
2	customer_unique_id	STRING		
3	customer_zip_code_prefix	INT64		
4	customer_city	STRING		
5	customer_state	STRING		

Insights

- The “customers” table has relatively low no of columns(features) compared to other columns and provides only necessary information related to customer’s information.
- As in above the output , except zipcode all others are in STRING format.
- The table does not have any constraints defined(i.e primary key and foreign key) at this moment

1.2 Get the time range between which the orders were placed.

SQL Query

```
SELECT MIN(order_purchase_timestamp) AS start_date,  
       MAX(order_purchase_timestamp) AS end_date  
FROM `sathish-scaler-projects.Target_DB.orders`;
```

Output

Query results			
Job information Results Chart JSON Execution details Exe			
Row	start_date ▾	end_date ▾	
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC	

Insights

- The data set has captured purchase order details from 4th September 2016 till 17th October 2018
- Since there are few dates missed in Sep 2016 (4days) and Oct 2018(14 days) , these months should be considered as outliers in-terms of any insights to be taken. Rest of the months in-between can be considered for analysis.

1.3 Count the Cities & States of customers who ordered during the given period.

SQL Query

```
SELECT  
  COUNT(DISTINCT customer_city) as city_count,  
  COUNT(DISTINCT customer_state) as state_count  
FROM `sathish-scaler-projects`.`Target_DB`.`orders` AS orders  
INNER JOIN `sathish-scaler-projects`.`Target_DB`.`customers` AS customers  
ON orders.customer_id = customers.customer_id  
WHERE  
--- Giving an Example where i can provide a time range here  
orders.order_purchase_timestamp BETWEEN '2017-01-01' AND '2018-01-01'
```

Output

Query results			
Job information Results Chart JSON Execution details			
Row	city_count ▾	state_count ▾	
1	3290	27	

Insights

- The data set has captured details of orders that is spread across 27 states and 3290 cities in the region of Brazil.
- I have added a where statement to provide during a time range from which city/states the orders are coming from.

2. In-depth Exploration:

2.1 Is there a growing trend in the no. of orders placed over the past years?

SQL Query

```
WITH find_trend AS
(
  -- Primary Query , this is sufficient to find the order count
  SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
    COUNT(*) AS orders_placed
  FROM `sathish-scaler-projects`.`Target_DB`.`orders`
  GROUP BY order_year
)
-- Adding this to indicate the percentage of increase
SELECT
  order_year,
  orders_placed,
  round((orders_placed - LAG(orders_placed, 1, 0) OVER (ORDER BY order_year)) *
100.0 /
    NULLIF(LAG(orders_placed, 1) OVER (ORDER BY order_year), 0),2) AS
order_increase_percentage
FROM find_trend
ORDER BY order_year;
```

Output

Query results

Job information	Results	Chart	JSON	Execution details
Row	order_year	orders_placed	order_increase_percentage	
1	2016	329	null	
2	2017	45101	13608.51	
3	2018	54011	19.76	

Insights

- The data set has captured details of orders that is spread across 27 states and 3290 cities in the region of Brazil.
- I have added a where statement to provide during a time range from which city/states the orders are coming from.

2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

SQL Query

```
WITH orders_with_year AS
(
  SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
    EXTRACT(MONTH FROM order_purchase_timestamp) AS order_month,
    COUNT(*) AS order_count
  FROM
    `sathish-scaler-projects`.`Target_DB`.`orders`
  GROUP BY order_year, order_month
)
SELECT
  order_year,
  order_month,
  order_count,
  CASE
    WHEN order_count > ( SELECT AVG(order_count) FROM `orders_with_year`) THEN 'High'
    ELSE 'Low'
  END
  AS order_volumes
FROM `orders_with_year`
ORDER BY order_year, order_month;
```

Output

Query results

Job information		Results	Chart	JSON	Execution details	Execution graph
Row	order_year	order_month	order_count	order_volumes		
1	2016	9	4	Low		
2	2016	10	324	Low		
3	2016	12	1	Low		
4	2017	1	800	Low		
5	2017	2	1780	Low		
6	2017	3	2682	Low		
7	2017	4	2404	Low		
8	2017	5	3700	Low		
9	2017	6	3245	Low		
10	2017	7	4026	High		
11	2017	8	4331	High		
12	2017	9	4285	High		
13	2017	10	4631	High		
14	2017	11	7544	High		
15	2017	12	5673	High		
16	2018	1	7269	High		
17	2018	2	6728	High		
18	2018	3	7211	High		
19	2018	4	6939	High		
20	2018	5	6873	High		
21	2018	6	6167	High		
22	2018	7	6292	High		
23	2018	8	6512	High		
24	2018	9	16	Low		
25	2018	10	4	Low		

Insights

- The 2016 September and 2018 Oct will be left as outlier as there are few dates are missing with details
- Order count increased in 2017 and 2018, compared to previous year, i.e 2016
- High order volumes account for approx. 85% of the total order count while low order volumes represent 15%.

2.3 During what time of the day, do the Brazilian customers mostly place their orders?

(Dawn, Morning, Afternoon or Night)

0-6 hrs : Dawn

7-12 hrs : Mornings

13-18 hrs : Afternoon

19-23 hrs : Night

SQL Query 1 (Just specifying also the minutes in-order not to miss certain null values)

```
WITH
classify_time AS (
SELECT
order_id,
CASE
WHEN TIME(order_purchase_timestamp) BETWEEN '00:00:00' AND '06:59:59' THEN 'DAWN'
WHEN TIME(order_purchase_timestamp) BETWEEN '07:00:00' AND '12:00:00' THEN 'MORNINGS'
WHEN TIME(order_purchase_timestamp) BETWEEN '12:00:01' AND '18:59:59' THEN 'AFTERNOON'
WHEN TIME(order_purchase_timestamp) BETWEEN '19:00:00' AND '23:59:59' THEN 'NIGHT'
END
AS purchase_time
FROM `sathish-scaler-projects`.`Target_DB`.`orders` )
SELECT
purchase_time,
COUNT(*) AS total_orders
FROM `classify_time`
GROUP BY purchase_time
ORDER BY total_orders DESC;
```

Output1 (In this output I have taken care of all null values by specifying the minutes)

Query results			
Job information		Results	Chart
Results		JSON	Execution details
Execution details		Execution grap	
Row	purchase_time ▼	total_orders ▼	
1	AFTERNOON	44130	
2	NIGHT	28331	
3	MORNINGS	21738	
4	DAWN	5242	

Insights

- Customers are busy ordering items during Afternoon,
- Afternoon sees the highest total orders, approx. 40% of the total.
- Customers don't purchase much during Dawn, as Dawn experiences the lowest total orders, approx 5% of the total.

3. Evolution of E-commerce orders in the Brazil region:

3.1 Get the month on month no. of orders placed in each state.

SQL Query

```
WITH CTE1 AS
(
    SELECT
        o.order_id,
        o.order_purchase_timestamp,
        EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
        EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
        FORMAT_DATE('%b %Y', DATE(ORDER_PURCHASE_TIMESTAMP)) AS time_period,
        c.customer_state AS state
    FROM `sathish-scaler-projects`.`Target_DB`.`orders` AS o
    INNER JOIN `sathish-scaler-projects`.`Target_DB`.`customers` AS c
    ON o.customer_id = c.customer_id
)
SELECT
    state,
    time_period,
    COUNT(*) AS total_orders,
    LAG(COUNT(*)) OVER (PARTITION BY state ORDER BY year, month) AS
prev_month_orders_count,
    ROUND( ( (COUNT(*) - LAG(COUNT(*)) OVER (PARTITION BY state ORDER BY year, month))
/ LAG(COUNT(*)) OVER (PARTITION BY state ORDER BY year, month)) * 100, 2) AS
MoM_percent_growth
FROM `CTE1`
GROUP BY state, time_period, year, month
ORDER BY state;
```

Output

Query results						
Job information		Results	Chart	JSON	Execution details	Execution graph
Row	state	time_period	total_orders	prev_month_orders_count	MoM_percent_growth	
1	AC	Jan 2017	2	null	null	
2	AC	Feb 2017	3	2	50.0	
3	AC	Mar 2017	2	3	-33.33	
4	AC	Apr 2017	5	2	150.0	
5	AC	May 2017	8	5	60.0	
6	AC	Jun 2017	4	8	-50.0	
7	AC	Jul 2017	5	4	25.0	
8	AC	Aug 2017	4	5	-20.0	
9	AC	Sep 2017	5	4	25.0	
10	AC	Oct 2017	6	5	20.0	
11	AC	Nov 2017	5	6	-16.67	
12	AC	Dec 2017	5	5	0.0	
13	AC	Jan 2018	6	5	20.0	
14	AC	Feb 2018	3	6	-50.0	
15	AC	Mar 2018	2	3	-33.33	
16	AC	Apr 2018	4	2	100.0	

Insights

- The order details are given as per each state over the period of time
- Calculated % of Month on Month growth by comparing to the previous month using LAG function to provide better visibility
- Also added previous month in the output to provide comparison and calculation of the output

3.2 How are the customers distributed across all the states?

SQL Query

```
SELECT
  DISTINCT customer_state,
  count(customer_unique_id) AS No_of_unique_customers

FROM `sathish-scaler-projects.Target_DB.customers`
GROUP BY customer_state
ORDER BY No_of_unique_customers DESC;
```

Output

Query results

Job information		Results	Chart	JSON	Execution details
Row	customer_state ▾	No_of_unique_custor			
1	SP	41746			
2	RJ	12852			
3	MG	11635			
4	RS	5466			
5	PR	5045			
6	SC	3637			
7	BA	3380			
8	DF	2140			
9	ES	2033			
10	GO	2020			
11	PE	1652			
12	CE	1336			
13	PA	975			
14	MT	907			
15	MA	747			
16	MS	715			
17	PB	536			
18	PI	495			
19	RN	485			
20	AL	413			
21	SE	350			

Load more

Insights

- The Brazil's Capital region SP-Sao Paulo and surrounding RJ - Rio de Janeiro RJ and MJ - states has more customers compared to other states
- The SP account for 42% of unique customers compared to overall customers
- RR has the lowest customer acquisition accounting for the least among unique customers
- The average no of customers per state is ~3683

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

4.1 Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

SQL Query

```
WITH orders_with_payment AS (
    SELECT
        o.order_id,
        EXTRACT(YEAR FROM o.order_purchase_timestamp) AS Year,
        EXTRACT(MONTH FROM o.order_purchase_timestamp) AS Month,
        p.payment_value
    FROM `sathish-scaler-projects`.`Target_DB`.`orders` AS o
    INNER JOIN `sathish-scaler-projects`.`Target_DB`.`payments` AS p
    ON o.order_id = p.order_id
    WHERE o.order_status = 'delivered'
),
monthly_total_payment AS (
    SELECT
        Year,
        Month,
        ROUND(SUM(payment_value),2) AS total_payment
    FROM orders_with_payment
    WHERE Month BETWEEN 1 AND 8
    GROUP BY Year, Month
),
yearly_total_payment AS (
    SELECT
        Year,
        ROUND(SUM(total_payment),2) AS yearly_total_payment
    FROM monthly_total_payment
    GROUP BY Year
)
SELECT
    y.Year,
    y.yearly_total_payment,
    COALESCE(
        ROUND(((y.yearly_total_payment - LAG(y.yearly_total_payment) OVER (ORDER BY
y.Year))
        / LAG(y.yearly_total_payment) OVER (ORDER BY y.Year)) * 100, 2),
        0
    ) AS YOY_percent_increase
FROM yearly_total_payment AS y
ORDER BY y.Year;
```

Query results

Job information		Results	Chart	JSON	Execution details
Row	Year ▼	yearly_total_payment ▼	YOY_percent_increase ▼		
1	2017	3473862.76	0.0		
2	2018	8452975.2	143.33		

Insights

- The Brazil’s Capital region SP-Sao Paulo and surrounding RJ - Rio de Janeiro RJ and MJ - states has more customers compared to other states
- The SP account for 42% of unique customers compared to overall customers
- RR has the lowest customer acquisition accounting for the least among unique customers
- The average no of customers per state is ~3683

4.2 Calculate the Total & Average value of order price for each state.

SQL Query

```
WITH payment_collection as
(
    SELECT
        c.customer_state,
        p.payment_value
    FROM `sathish-scaler-projects.Target_DB.payments` p
    JOIN `sathish-scaler-projects.Target_DB.orders` o
    USING(order_id)
    JOIN `sathish-scaler-projects.Target_DB.customers` c
    USING(customer_id)
)
SELECT
    DISTINCT customer_state,
    ROUND(AVG(payment_value) OVER (partition by customer_state),2) as
avg_state_order_price,
    ROUND(SUM(payment_value) OVER (partition by customer_state),2) as
total_state_order_price
FROM payment_collection
ORDER BY total_state_order_price desc ,avg_state_order_price desc, customer_state
```

Output

Query results

Job information		Results	Chart	JSON	Execution details
Row	customer_state ▼	avg_state_order_pric	total_state_order_pri		
1	SP	137.5	5998226.96		
2	RJ	158.53	2144379.69		
3	MG	154.71	1872257.26		
4	RS	157.18	890898.54		
5	PR	154.15	811156.38		
6	SC	165.98	623086.43		
7	BA	170.82	616645.82		
8	DF	161.13	355141.08		
9	GO	165.76	350092.31		
10	ES	154.71	325967.55		
11	PE	187.99	324850.44		
12	CE	199.9	279464.03		
13	PA	215.92	218295.85		
14	MT	195.23	187029.29		
15	MA	198.86	152523.02		
16	PB	248.33	141545.72		
17	MS	186.87	137534.84		
18	PI	207.11	108523.97		
19	RN	196.78	102718.13		
20	AL	227.08	96962.06		
21	SE	208.44	75246.25		
22	TO	204.27	61485.33		
23	RO	233.2	60866.2		
24	AM	181.6	27966.93		
25	AC	234.29	19680.62		
26	AP	232.33	16262.8		
27	RR	218.8	10064.62		

SQL Query

- There is an inverse relationship between average order price and total order price by state
- Few states such as PB, AL ,PA has better average price compared to the highest order value
- SP – Sao Paulo has topped with total no of sales and sales volume. Due to volume of trade customers also benefit with reduced price in their purchases.

4.3 Calculate the Total & Average value of order freight for each state.

SQL Query

```
SELECT
  c.customer_state,
  ROUND(SUM(i.price)) total_price,           -- calculating total for price value
  ROUND(AVG(i.price)) avg_price,             -- calculating avg for price value
  ROUND(SUM(i.freight_value)) total_freight_value, -- calc total for freight value
  ROUND(AVG(i.freight_value)) avg_freight_value -- calculating avg for freight value
FROM `Target_DB.items` i
JOIN `Target_DB.orders` o
  USING (order_id)
JOIN `Target_DB.customers` c
  ON c.customer_id = o.customer_id
GROUP BY c.customer_state;
```

Output

Query results						
Job information		Results	Chart	JSON	Execution details	Execution graph
Row	customer_state	total_price	avg_price	total_freight_value	avg_freight_value	
1	SP	5202955.0	110.0	718723.0	15.0	
2	RJ	1824093.0	125.0	305589.0	21.0	
3	PR	683084.0	119.0	117852.0	21.0	
4	RS	750304.0	120.0	135523.0	22.0	
5	MG	1585308.0	121.0	270853.0	21.0	
6	GO	294592.0	126.0	53115.0	23.0	
7	SC	520553.0	125.0	89660.0	21.0	
8	PB	115268.0	191.0	25720.0	43.0	
9	DF	302604.0	126.0	50626.0	21.0	
10	PA	178948.0	166.0	38699.0	36.0	
11	BA	511350.0	135.0	100157.0	26.0	
12	ES	275037.0	122.0	49765.0	22.0	
13	PE	262788.0	146.0	59450.0	33.0	
14	MS	116813.0	143.0	19144.0	23.0	
15	MT	156454.0	148.0	29715.0	28.0	
16	MA	119648.0	145.0	31524.0	38.0	
17	CE	227255.0	154.0	48352.0	33.0	
18	TO	49622.0	158.0	11733.0	37.0	
19	AM	22357.0	135.0	5479.0	33.0	
20	AL	80315.0	181.0	15915.0	36.0	
21	RO	46141.0	166.0	11417.0	41.0	
22	RN	83035.0	157.0	18860.0	36.0	
23	PI	86914.0	160.0	21218.0	39.0	
24	AC	15983.0	174.0	3687.0	40.0	
25	SE	58921.0	153.0	14111.0	37.0	
26	RR	7829.0	151.0	2235.0	43.0	
27	AP	13474.0	164.0	2788.0	34.0	

Insights

- There's a positive correlation between total price and total freight value, as well as between average price and average freight value.
- SP leads in both total price and total freight value and value of the total, respectively.
- RR has the lowest values in both total price and total freight value, in both categories.
- While PB has the highest average price, SP has the lowest and also the lowest average freight value.

5. Analysis based on sales, freight and delivery time.

5.1 Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- **time_to_deliver** = order_delivered_customer_date - order_purchase_timestamp.
- **diff_estimated_delivery** = order_estimated_delivery_date - order_delivered_customer_date

SQL Query

```
SELECT
    orders.order_id,
    DATE_DIFF( DATE(orders.order_delivered_customer_date),
DATE(orders.order_purchase_timestamp), DAY) AS time_to_deliver_days,
    DATE_DIFF( DATE(orders.order_estimated_delivery_date),
DATE(orders.order_delivered_customer_date), DAY) AS diff_estimated_delivery_days
FROM `sathish-scaler-projects.Target_DB.orders` AS orders
WHERE
    orders.order_delivered_customer_date IS NOT NULL
    AND orders.order_purchase_timestamp IS NOT NULL;
```

Output

Query results

Job information		Results	Chart	JSON	Execution details	Execution graph
Row	order_id	time_to_deliver_days	diff_estimated_delive			
1	65d1e226dfaeb8cdc42f66542...	36	17			
2	2c45c33d2f9cb8ff8b1c86cc28...	31	29			
3	1950d777989f6a877539f5379...	30	-12			
4	bfb0f9bdef84302105ad712d...	55	-36			
5	98974b076b01553d49ee6467...	44	7			
6	c4b41c36dd589e901f6879f25...	36	15			
7	d2292ff2201e74c5db154d1b7...	30	21			
8	95e01270fcb9e986342340010...	31	20			
9	ed8c7b1b3eb256c70ce0c7423...	45	6			
10	5cc475c7c03290048eb2e742c...	69	-18			
11	6b3ee7697a02619a0ace2b3f0...	48	3			
12	3b2ca3293a7ce539ea2379d70...	43	8			
13	b2f92b2f7047cd8b35580d629...	43	8			
14	e2eaf909eb6ba881117aa4079...	40	11			
15	90aea7c4e52538a18cb9bbfd1...	44	11			
16	dd11631d8b02780b78bd97ec...	31	24			
17	f781dacd75aa73166785a3592...	33	20			
18	b0cfee6ce5503781be6004e0e...	56	-1			
19	893dc49276696129a9bf6f2a9f...	30	25			
20	bb9fd560a08ca7403789dedac...	32	23			
21	63638a6806d67773f3adba853...	52	3			
22	5bda3e4d7080c484cd646a531...	40	13			
23	5da2004d85f2828349d887c98...	52	3			
24	9315757eaf0e782619bed1315...	31	22			
25	2d9e3c3c7f0f3ba8a8fa3db2f1...	33	22			
26	a41c8759fbe7aab36ea07e038...	31	26			
27	8d47519f5c9c42540b37bd3db...	34	23			

Insights

- The average time to deliver is 12.29 days, while the average difference from estimated delivery is 7.69 days.
- There are outliers like few order ID (ca07593549f1816d26a572e06dc1eab6) has taken 210 days likely due to incorrect information provided during delivery.
- There are instances also where some of the delivery has been done on the same day (0 days)
- There are some information not captured properly resulting in negative values, this requires proper data input and cleanup.

5.2 Find out the top 5 states with the highest & lowest average freight value.

SQL Query

```
WITH StateAvgFreight AS --1st query to find avg_freight cost
(
    SELECT
        customers.customer_state,
        AVG(items.freight_value) AS avg_freight
    FROM `sathish-scaler-projects.Target_DB.customers` AS customers
    INNER JOIN `sathish-scaler-projects.Target_DB.orders` AS orders
    ON customers.customer_id = orders.customer_id
    INNER JOIN `sathish-scaler-projects.Target_DB.items` AS items
    ON orders.order_id = items.order_id
    GROUP BY
        customers.customer_state
),
RankedStates AS --2nd Query to rank the states, output same as DENSE_RANK
(
    SELECT
        customer_state,
        avg_freight,
        RANK() OVER (ORDER BY avg_freight DESC) AS rank_highest,
        RANK() OVER (ORDER BY avg_freight ASC) AS rank_lowest
    FROM `StateAvgFreight`
)
SELECT
    'Highest FV' AS Category,
    customer_state,
    avg_freight
FROM `RankedStates`
WHERE rank_highest <= 5 -- To find top 5 from ranked states (ASC)
UNION ALL -- Combining Highest with lowest to see in single output
SELECT
    'Lowest FV' AS Category,
    customer_state,
    avg_freight
FROM `RankedStates`
WHERE rank_lowest <= 5 -- To find lowest 5 from ranked (DESC)
ORDER BY
    Category,
    avg_freight DESC;
```


Output

Query results

Job information		Results	Chart	JSON	Execution details	Execution graph
Row	Category	customer_state	avg_freight			
1	Highest FV	RR	42.98442307692...			
2	Highest FV	PB	42.72380398671...			
3	Highest FV	RO	41.06971223021...			
4	Highest FV	AC	40.07336956521...			
5	Highest FV	PI	39.14797047970...			
6	Lowest FV	DF	21.04135494596...			
7	Lowest FV	RJ	20.96092393168...			
8	Lowest FV	MG	20.63016680630...			
9	Lowest FV	PR	20.53165156794...			
10	Lowest FV	SP	15.14727539041...			

Insights

- For the Highest Freight Value category, RR has the highest average freight at 42.98, while PI has the lowest at 39.15
- Conversely, for the Lowest Frieght Value category, DF exhibits the highest average freight at 21.04, and SP the lowest at 15.15.
- The average freight varies significantly by category, with a range of 27.83 between the highest (RR in Highest FV) and lowest (SP in Lowest FV) values.

5.3 Find out the top 5 states with the highest & lowest average delivery time.

SQL Query

```
WITH StateAvgDeliverytime AS
(
    SELECT
        c.customer_state,
        AVG(TIMESTAMP_DIFF(order_delivered_customer_date, order_purchase_timestamp,
DAY)) AS avg_delivery_time
    FROM `sathish-scaler-projects.Target_DB.customers` c
    INNER JOIN `sathish-scaler-projects.Target_DB.orders` o
    ON c.customer_id = o.customer_id
    GROUP BY c.customer_state
),
RankedStates AS
(
    SELECT
        customer_state,
        avg_delivery_time,
        DENSE_RANK() OVER (ORDER BY avg_delivery_time DESC) AS rank_highest,
        DENSE_RANK() OVER (ORDER BY avg_delivery_time) AS rank_lowest
    FROM StateAvgDeliverytime
)
SELECT
    'Highest' AS Category,
    customer_state,
    avg_delivery_time
FROM RankedStates
WHERE rank_highest <= 5
UNION ALL
SELECT
    'Lowest' AS Category,
    customer_state,
    avg_delivery_time
FROM RankedStates
WHERE rank_lowest <= 5
ORDER BY Category, avg_delivery_time DESC;
```

Query results

Job information		Results	Chart	JSON	Execution details	Execu
Row	Category	customer_state	avg_delivery_time			
1	Highest	RR	28.97560975609...			
2	Highest	AP	26.73134328358...			
3	Highest	AM	25.98620689655...			
4	Highest	AL	24.04030226700...			
5	Highest	PA	23.31606765327...			
6	Lowest	SC	14.47956019171...			
7	Lowest	DF	12.50913461538...			
8	Lowest	MG	11.54381329810...			
9	Lowest	PR	11.52671135486...			
10	Lowest	SP	8.298061489072...			

Insights

- Average delivery time varies significantly by customer state and category due to proximity. For better delivery times we need local distribution and warehouses for states like RR, AP AM , AL and PA.
- For the "Highest" category, RR has the longest average delivery time at 28.98, while PA has the shortest at 23.32, a difference of 19.5%.
- Conversely, for the "Lowest" category, SC has the longest average delivery time at 14.48, while SP has the shortest at 8.3, a difference of 42.7%. This indicates the delivery and distribution network is better

5.4 Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

SQL Query

```
WITH DeliveryRanking AS
(
  SELECT
    c.customer_state,
    AVG( DATE_DIFF( o.order_estimated_delivery_date,o.order_delivered_customer_date,
DAY)) AS average_delivery_time_difference,
    RANK() OVER (ORDER BY AVG(DATE_DIFF(o.order_delivered_customer_date,
o.order_estimated_delivery_date, DAY)) DESC ) AS delivery_ranking
  FROM `sathish-scaler-projects.Target_DB.orders` o
  LEFT JOIN `sathish-scaler-projects.Target_DB.customers` c
  ON o.customer_id = c.customer_id
  WHERE
    o.order_delivered_customer_date IS NOT NULL
    AND o.order_estimated_delivery_date IS NOT NULL
  GROUP BY c.customer_state
)
SELECT
  customer_state,
  average_delivery_time_difference,
  delivery_ranking
FROM DeliveryRanking
WHERE delivery_ranking <= 5
ORDER BY delivery_ranking ASC;
```

Output

Query results

Job information		Results	Chart	JSON	Execution details	Exec
Row	customer_state ▾	average_delivery_tim	delivery_ranking ▾			
1	AL	7.947103274559...	1			
2	MA	8.768479776847...	2			
3	SE	9.173134328358...	3			
4	ES	9.618546365914...	4			
5	BA	9.934889434889...	5			

Insights

- A positive relationship exists between the average delivery time difference and the delivery ranking.
- AL boasts the fastest average delivery time difference at around 7.95 days and secures the highest delivery ranking.
- Delivery ranking declines as the average delivery time difference increases, culminating in BA's lowest ranking with an average of approximately 9.93 days.
- The gap in average delivery time between the top-ranked AL and the bottom-ranked BA is roughly 1.98 days.

6. Analysis based on the payments:

6.1 Find the month on month no. of orders placed using different payment types.

SQL Query

```
SELECT
  FORMAT_TIMESTAMP('%Y-%m', o.order_purchase_timestamp)as period,
  COUNT(case when payment_type='credit_card' then 1 end) as credit_card,
  COUNT(case when payment_type='debit_card' then 1 end)as debit_card,
  COUNT(case when payment_type='UPI' then 1 end) as UPI,
  COUNT(case when payment_type='voucher' then 1 end) as voucher,
  COUNT(case when payment_type='not_defined' then 1 end)as not_defined,
  COUNT(case when payment_type is null then 1 end)as not_known,
  COUNT(o.order_id)as total_orders
FROM `Target_DB.payments` p
JOIN `Target_DB.orders` o
ON p.order_id=o.order_id
WHERE payment_type is not null and order_status not in ('canceled','unavailable')
GROUP BY period
ORDER BY period;
```

Output

Query results										
Job information		Results	Chart	JSON	Execution details		Execution graph			
Row	period	credit_card	debit_card	UPI	voucher	not_defined	not_known	total_orders		
1	2016-09	1	0	0	0	0	0	1		
2	2016-10	227	2	60	22	0	0	311		
3	2016-12	1	0	0	0	0	0	1		
4	2017-01	574	9	193	60	0	0	836		
5	2017-02	1309	13	383	116	0	0	1821		
6	2017-03	1959	30	582	200	0	0	2771		
7	2017-04	1828	26	488	201	0	0	2543		
8	2017-05	2807	30	758	288	0	0	3883		
9	2017-06	2428	27	702	236	0	0	3393		
10	2017-07	3024	22	829	349	0	0	4224		
11	2017-08	3244	34	920	275	0	0	4473		
12	2017-09	3243	43	885	279	0	0	4450		
13	2017-10	3466	52	971	280	0	0	4769		
14	2017-11	5812	67	1479	382	0	0	7740		
15	2017-12	4333	63	1152	291	0	0	5839		
16	2018-01	5457	109	1499	412	0	0	7477		
17	2018-02	5167	68	1311	300	0	0	6846		
18	2018-03	5658	77	1343	390	0	0	7468		
19	2018-04	5438	97	1286	368	0	0	7189		
20	2018-05	5465	50	1257	316	0	0	7088		
21	2018-06	4795	181	1097	324	0	0	6397		
22	2018-07	4714	240	1212	280	0	0	6446		
23	2018-08	4955	276	1132	243	0	0	6606		
24	2018-09	0	0	0	1	0	0	1		

Insights

- Total orders experienced a significant decline of 99.98% from August 2018 to September 2018.
- All payment methods (credit card, debit card, and UPI) decreased by 100% between August and September 2018.
- A positive correlation exists between credit card usage and debit card usage, as well as between total orders and credit card usage.
- Total orders peaked at 7740 in November 2017, while credit card transactions reached a maximum of 5812 during the same period.

6.2 Find the no. of orders placed on the basis of the payment instalments that have been paid.

SQL Query

```
SELECT
  p.payment_installments,
  COUNT(o.order_id) as total_orders
FROM `sathish-scaler-projects.Target_DB.orders` o
JOIN `sathish-scaler-projects.Target_DB.payments` p
USING (order_id)
GROUP BY p.payment_installments
ORDER BY p.payment_installments;
```

Output

Query results

Job information	Results	Chart	JSON	Execution details	Execution graph
Row	payment_installment	total_orders			
1	0	2			
2	1	52546			
3	2	12413			
4	3	10461			
5	4	7098			
6	5	5239			
7	6	3920			
8	7	1626			
9	8	4268			
10	9	644			
11	10	5328			
12	11	23			
13	12	133			
14	13	16			
15	14	15			
16	15	74			
17	16	5			
18	17	8			
19	18	27			
20	20	17			
21	21	3			
22	22	1			
23	23	1			
24	24	18			

Insights

- A large number of orders are paid in a single instalment, indicating a preference for immediate payment.
- The frequency of orders decreases as the number of instalments increases from 2 to 6, suggesting a preference for shorter payment plans.
- There is a sharp decline in orders using more than 6 instalments, showing limited adoption of extended payment plans.

E. Overall Insights – Key takeaways

Customer Distribution & Trends:

The highest number of unique customers are concentrated in SP (Sao Paulo) and surrounding regions. There's a consistent increase in order volume over the years, highlighting growing e-commerce adoption.

Seasonality & Purchase Timing:

Orders exhibit monthly seasonality, with peak volumes in certain months. Brazilian customers predominantly place orders in the afternoon, accounting for approximately 40% of total purchases.

Economic Impact & Payment Behaviour:

There was a notable **increase in e-commerce transaction value** from 2017 to 2018. Single-installment payments dominate, with fewer orders using extended payment plans, signalling consumer preferences for upfront transactions.

Freight & Delivery Optimization:

Sao Paulo benefits from reduced freight costs due to high trade volume, while RR has the highest freight rates. Delivery speeds vary significantly, with states like AL experiencing the fastest order fulfillment compared to estimated delivery dates.

Operational Improvements & Outliers:

Some orders took exceptionally long to deliver, indicating potential logistical issues. Cleaning up data inaccuracies (e.g., negative delivery time values) will improve reporting reliability and strategic insights.

Regional Expansion & Customer Acquisition:

High customer concentration in São Paulo and Rio de Janeiro suggests strong demand. Expanding localized marketing efforts and fulfillment centres in emerging regions like RR and PB can boost growth.

Freight & Logistics Partnerships:

Freight costs vary significantly across states. Partnering with local carriers and warehouses to reduce delivery time and cost, especially in high-freight states like RR, can improve margins.

Delivery Optimization & Outlier Mitigation:

Orders with long delivery times and negative values indicate logistical issues or data inaccuracies. Streamlining delivery routes and improving order tracking accuracy will minimize inefficiencies.